

AMSC_STFT

A parallel C++ library for the Short-Time Fourier Transform

Marta Arzeni

Matilde Colombo

Academic year 2025/2026

Abstract

AMSC_STFT is a C++20 library for the spectral analysis of audio signals, built around the Short-Time Fourier Transform (STFT). It covers the whole pipeline, from reading a WAV file to exporting a time–frequency spectrogram as an image, and provides three FFT implementations, a choice of window functions (Hann, Hamming, Blackman), and two levels of parallelism: OpenMP for shared memory and MPI for distributed memory, usable together in a hybrid configuration. This report describes the mathematical problem, the organisation of the repository, the design decisions behind the architecture, the parallelisation strategy, and the performance measured on the Galileo100 cluster at CINECA, on a single node and on a four-node allocation of 192 cores.

Contents

1	The mathematical problem	2
1.1	From the DFT to the STFT	2
1.2	Frame length and hop size	2
1.3	Windowing and spectral leakage	3
1.4	Spectrum representation and normalisation	3
2	Spectrogram examples	5
3	Organisation of the repository	7
3.1	Repository structure	7
3.2	Repository overview	9
4	Design and architecture	9
4.1	Compile-time polymorphism with CRTP	9
4.2	Interface checking with C++20 concepts	10
4.3	FFT factory and independent parallelism levels	11
5	Parallelisation strategy	11
5.1	Level one: parallelism inside a single transform	11
5.2	Level two: parallelism across frames on one node	12
5.3	Level three: parallelism across nodes with MPI	13
5.4	Composition: hybrid parallelism and the MPI layer	13
6	Benchmarks	16
6.1	Methodology	16
6.2	The three FFT engines	16
6.3	Where to spend the threads: granularity	17
6.4	Strong scaling of the STFT	18
6.5	The limits of the MPI implementation	20

1 The mathematical problem

1.1 From the DFT to the STFT

The **Discrete Fourier Transform (DFT)** decomposes a finite signal into its frequency components. Given a real-valued signal $x[n] \in \mathbb{R}$ of length N ,

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-i2\pi kn/N}, \quad k = 0, 1, \dots, N-1. \quad (1)$$

Each coefficient $X[k]$ can be interpreted as the inner product between the signal and a complex exponential at frequency $k f_s / N$, and therefore quantifies the contribution of that frequency component to the signal. Its modulus is proportional to the amplitude of the corresponding frequency component, while its argument determines its phase. For computational efficiency, the DFT is implemented using the **Fast Fourier Transform (FFT)**, which reduces the computational complexity from $O(N^2)$ to $O(N \log N)$.

Since every sample contributes to every coefficient, the DFT describes which frequencies are present but not when they occur. This is not a problem for stationary signals, whose spectral content does not change over time, but it is important for audio, where the frequency content can vary continuously. A time–frequency representation is therefore needed.

The **Short-Time Fourier Transform (STFT)** obtains such a representation by applying the DFT locally. The signal is divided into short frames that may overlap, each multiplied by a window function $w[n]$ that smoothly tapers towards its edges, and a DFT is calculated for each frame:

$$X[m, k] = \sum_{n=0}^{L-1} x[n + mH] w[n] e^{-i2\pi kn/L} \quad (2)$$

L frame length

$w[n]$ window

H hop size (the shift between consecutive frames)

m frame index

The result is a matrix in which (m) indexes time and (k) indexes frequency. The magnitude ($|X[m, k]|$) quantifies the strength of the corresponding frequency component and can be displayed as the intensity of a pixel in a spectrogram.

1.2 Frame length and hop size

The **frame length** L determines both temporal and spectral resolution, in opposite directions. A frame of L samples at a sample rate f_s has a duration Δt , while the corresponding frequency-bin spacing Δf is given by

$$\Delta t = \frac{L}{f_s} \text{ seconds}, \quad \Delta f = \frac{f_s}{L} \text{ hertz}.$$

Since $\Delta t \cdot \Delta f = 1$ regardless of L , improving temporal resolution necessarily reduces the frequency-bin spacing, and vice versa. With the default parameters used throughout this project, $L = 1024$ and $f_s = 44,100$ Hz, each frame lasts approximately 23.2 ms, while adjacent frequency bins are spaced by approximately 43.1 Hz.

The choice of $L = 1024 = 2^{10}$ provides a practical compromise between temporal and spectral resolution and satisfies the power-of-two requirement of the radix-2 Cooley–Tukey FFT algorithm.

With a frequency-bin spacing of 43.1 Hz, this setting works well for notes in the middle and upper range of the piano, where adjacent notes are more widely separated in frequency. However, it is less suitable for distinguishing low-pitched notes, where the frequency differences between adjacent notes are smaller.

The **hop size** H is a separate parameter that controls the overlap between consecutive frames and how densely the signal is sampled along the time axis. With $H = L$, frames do not overlap, while $H = L/2$ or $H = L/4$ produces increasing overlap.

Given a signal of N samples, the **number of frames** M is

$$M = \begin{cases} 0, & N < L, \\ 1 + \left\lfloor \frac{N - L}{H} \right\rfloor, & N \geq L. \end{cases} \quad (3)$$

1.3 Windowing and spectral leakage

Without windowing, the DFT treats the frame as one period of a periodic signal, so a mismatch between its first and last samples introduces an artificial discontinuity whose energy spreads across the spectrum. A window smoothly reduces the signal towards the frame boundaries, limiting this effect, called *spectral leakage*. Different windows provide different trade-offs between frequency resolution and suppression of spectral leakage.

In the frequency domain, a window produces a main lobe, which is the central peak around the analysed frequency, together with smaller sidelobes around it:

- a narrower **main lobe** allows nearby frequencies to be distinguished more clearly;
- lower **sidelobes** reduce the amount of energy leaking into other frequencies.

The three windows implemented in this project are described in the following table:

Table 1: The three window functions used in the implementation. The equivalent noise bandwidth (ENBW) measures the effective bandwidth of the window in frequency bins. The coherent gain is the mean of the window coefficients and is used in the amplitude normalisation of Section 1.4.

Window	$w[n]$	Coh. gain	ENBW	Peak sidelobe	Typical use
Hann	$0.5(1 - \cos \theta_n)$	0.50	1.50	-31.5 dB	general purpose
Hamming	$0.54 - 0.46 \cos \theta_n$	0.54	1.36	-43 dB	speech
Blackman	$0.42 - 0.5 \cos \theta_n + 0.08 \cos 2\theta_n$	0.42	1.73	-58 dB	precision analysis

$$\theta_n = 2\pi n/(L - 1), \quad n = 0, \dots, L - 1.$$

The Hann window is used as the default because it provides a good general compromise between frequency resolution and sidelobe suppression. Hamming provides lower sidelobes, while Blackman offers even stronger sidelobe suppression at the cost of a wider main lobe.

1.4 Spectrum representation and normalisation

Since each frame is a real-valued signal, its DFT has Hermitian symmetry:

$$X[L - k] = \overline{X[k]}. \quad (4)$$

This means that the second half of the spectrum is a mirror of the first half: the two bins are complex conjugates. Therefore, only the first $L/2 + 1$ coefficients need to be stored, corresponding to frequencies from 0 to the frequency $f_s/2$ (called Nyquist).

The magnitude of the DFT must also be normalised before being stored in the spectrogram, for two reasons:

- the DFT used in Equation (1) is not normalised, so its magnitude depends on the frame length L ;
- multiplying the signal by a window changes its amplitude. This effect is described by the *coherent gain* of the window, defined as the mean of its coefficients:

$$\text{CG}(w) = \frac{1}{L} \sum_{n=0}^{L-1} w[n]. \quad (5)$$

To compensate for both the frame length and the window attenuation, the magnitude is therefore normalised as

$$\boxed{S[m, k] = \frac{|X[m, k]|}{L \cdot \text{CG}(w)}} \quad k = 0, \dots, L/2. \quad (6)$$

2 Spectrogram examples

The following spectrograms show some examples of the STFT and of the different window functions used in the project.

In all figures, time is shown on the horizontal axis and frequency on the vertical axis, from 0 to 22050 Hz. The brightness of each point represents the magnitude of the corresponding frequency component: brighter points indicate stronger components, darker points weaker ones.

Figure 1 shows two different types of signals and how they appear in the STFT. The sweep changes its frequency gradually over time, so it appears as a curved line in the spectrogram. The short transients, instead, are very short in time but contain many different frequencies, so they appear as vertical lines covering a large part of the frequency range.

The transients last only about 1.3 ms, but the STFT uses frames of $L = 1024$ samples, which correspond to about 23 ms at 44.1 kHz. Therefore, the transients appear spread over about 23 ms in the spectrogram. This illustrates the trade-off of the STFT: using a longer frame gives better frequency information, but makes short events less precise in time.

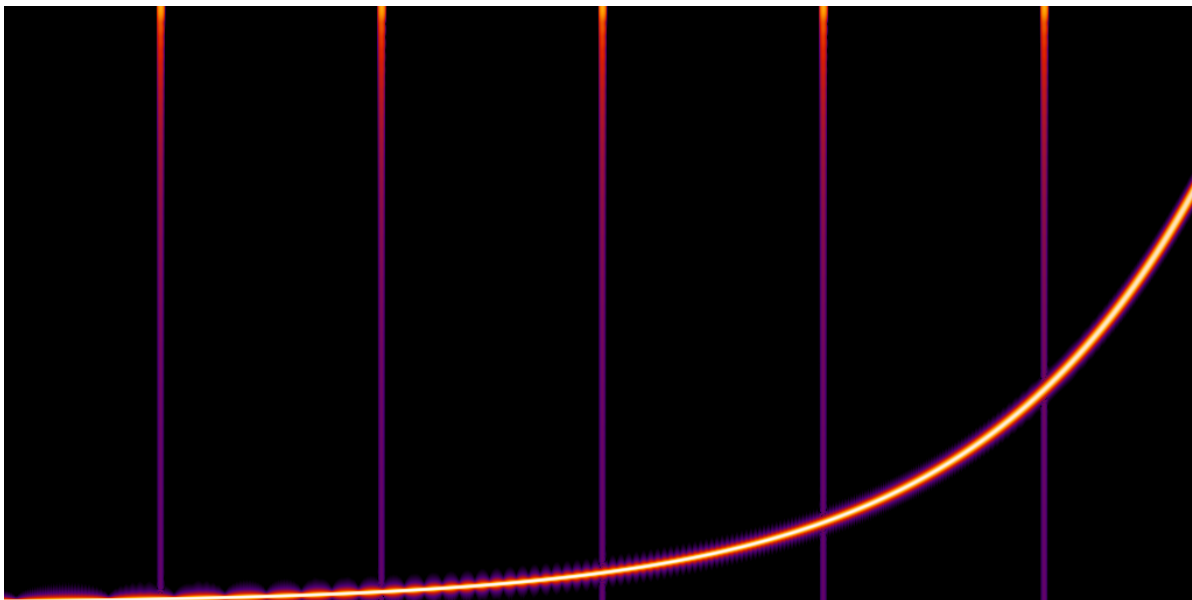
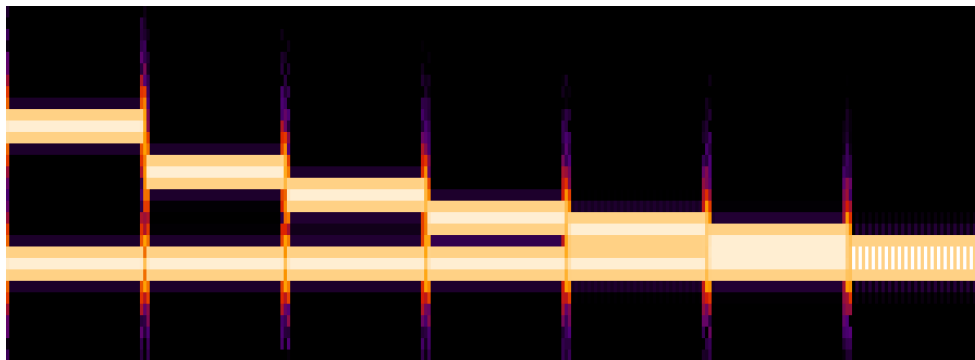


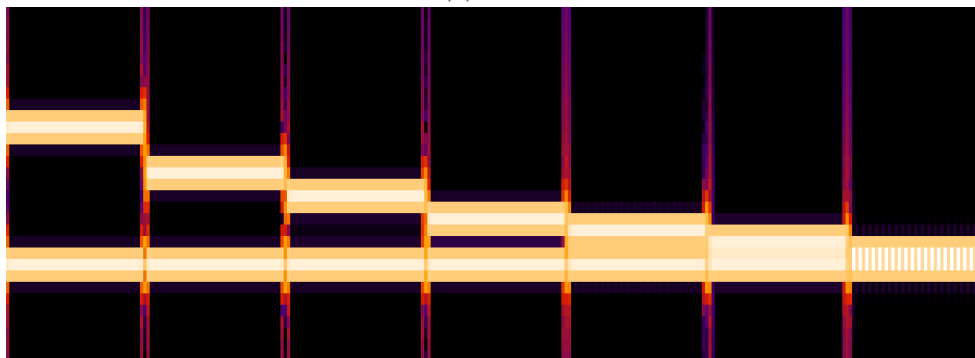
Figure 1: Exponential sweep from 80 Hz to 16 kHz with five short transients. The sweep is narrow in frequency and spread over time, while each transient is concentrated in time and spreads its energy over the whole frequency range.

Figure 2 shows the result of an experiment that compares the three window functions used in the project: Hann, Hamming and Blackman. Each window has a different effect on the frequency representation. In particular, as already observed in the previous section, reducing spectral leakage usually comes at the cost of a wider main lobe, which makes it harder to distinguish two close frequencies. To show this effect, two tones with the same amplitude are progressively moved closer to each other.

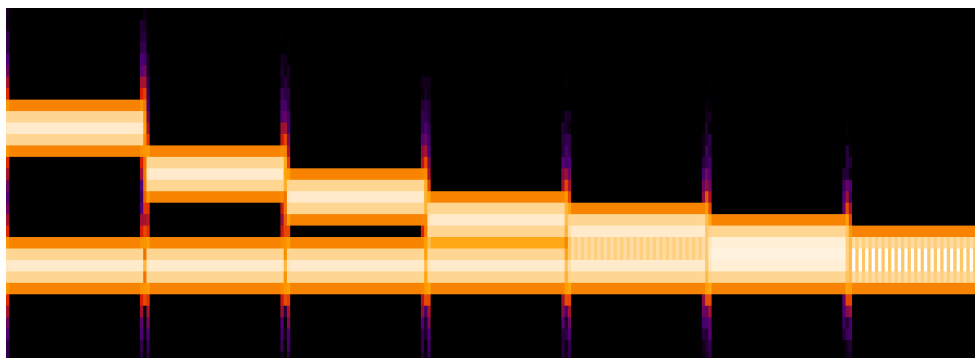
Hann and Hamming have a similar main-lobe width and therefore show a similar frequency resolution. Blackman has a wider main lobe, so two close tones merge into a single trace earlier than with the other two windows. This is the price paid for its lower spectral leakage: a strong component spreads less energy onto the surrounding frequencies, which keeps the rest of the spectrum cleaner.



(a) Hann



(b) Hamming



(c) Blackman

Figure 2: Frequency resolution of the three windows for two tones with different frequency separations.

Figure 3 shows a Shepard–Risset glissando. Ten partials move upwards together, while individual partials fade in at the bottom of the frequency range and fade out at the top. This creates the illusion of a pitch that keeps rising without ever reaching an end. The figure shows that the STFT can represent a complex signal with multiple frequency components changing at the same time.

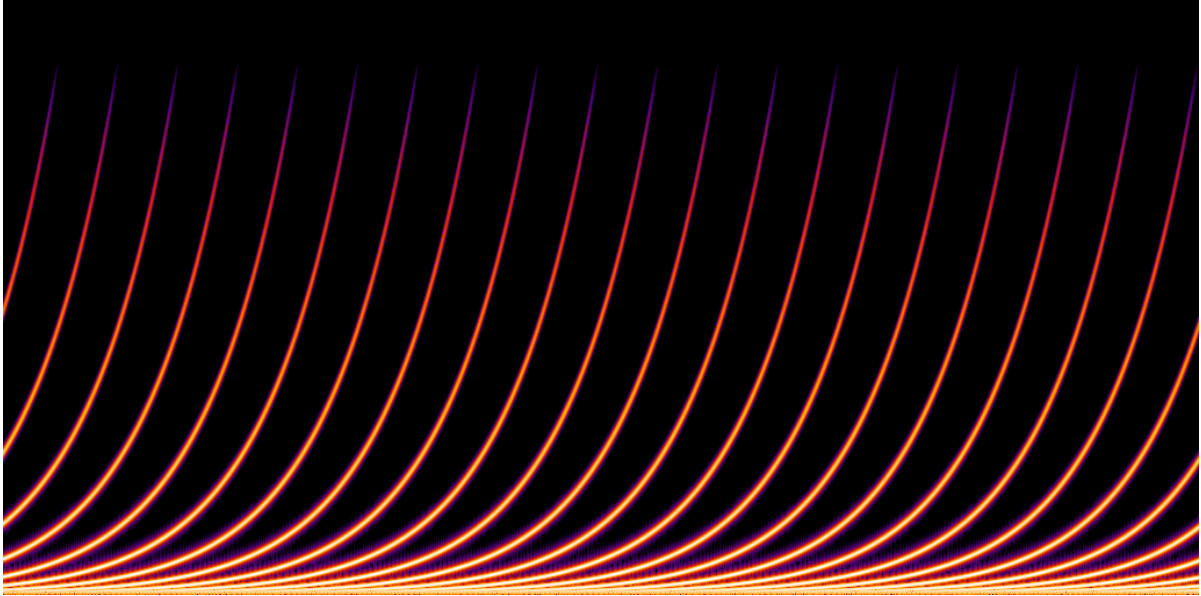


Figure 3: Shepard–Risset glissando with ten octave-spaced partials. The partials continuously enter and leave the frequency range, creating the illusion of an endlessly rising pitch.

3 Organisation of the repository

3.1 Repository structure

The repository is organised into four main areas: the C++ library, the scripts used to run it, the tools used to analyse the results, and the files generated during experiments. This separation keeps the source code independent from the execution environment and from the benchmark analysis.

The main structure of the repository is shown below.

```
AMSC_STFT/
|
|-- .github/
| +-- workflows/
| +-- main.yaml <- GitHub Actions CI pipeline
|
|-- system-deps.txt <- packages for the C++/MPI toolchain
|-- Singularity.def <- container definition for HPC
| deployment
|
|-- scripts/ <- scripts used to run the project
| |-- run_suite.sh <- benchmark and test suite
| |-- job.sh <- SLURM submission for CI/CD
|
|-- analysis/ <- tools used to analyse the results
| |-- parse_results.py <- merges and processes benchmark data
| |-- plot_results.py <- generates figures with matplotlib
| +-- requirements.txt <- Python dependencies
|
```

```

|-- core/ <- C++ library and applications
| |
| |-- CMakeLists.txt
| |
| |-- include/
| | |-- audio/
| | | |-- AudioFile.h
| | | +-- WavReader.hpp
| | |
| | |-- fft/
| | | |-- BaseFFT.hpp
| | | |-- IterativeFFT.hpp
| | | |-- RecursiveFFT.hpp
| | | +-- ParallelFFT.hpp
| | |
| | |-- window/
| | | |-- BaseWindow.hpp
| | | |-- HannWindow.hpp
| | | |-- HammingWindow.hpp
| | | +-- BlackmanWindow.hpp
| | |
| | |-- mpi/
| | | +-- MPIContext.hpp
| | |
| | |-- stft/
| | | |-- STFTAnalyzer.hpp
| | | |-- MPI_STFTAnalyzer.hpp
| | | +-- SpectrogramData.hpp
| | |
| | +-- output/
| | +-- ImageExporter.hpp
| |
|-- src/ <- C++ implementations
| |-- audio/
| | +-- WavReader.cpp
| |
| |-- fft/
| | |-- IterativeFFT.cpp
| | |-- RecursiveFFT.cpp
| | +-- ParallelFFT.cpp
| |
| |-- mpi/
| | +-- MPIContext.cpp
| |
| +-- output/
| +-- ImageExporter.cpp
| |
|-- tests/ <- automated tests
| |-- CMakeLists.txt
| +-- (test source files)
| |
|-- benchmarks/ <- performance benchmarks
| |-- CMakeLists.txt
| |-- benchmark_Suite.hpp
| |-- benchmark_Main.cpp
| +-- benchmark_MPI_Main.cpp
| |
+-- examples/ <- example applications
|-- CMakeLists.txt
|-- main.cpp
|-- mpi_main.cpp
+-- data/
|-- test_audio.wav
+-- scale.wav

+-- results/ <- generated files, not stored in git

```

```

|-- results_benchmark/ <- benchmark output
|-- results_test/ <- test logs and temporary test files
|-- results_analysis/ <- processed data and figures
| |-- csv/
| +-- figures/
+-- results_examples/ <- generated spectrograms

```

3.2 Repository overview

`core/` contains the main C++ code of the project: the audio input, FFT implementations, window functions, STFT analysis, MPI support, tests, benchmarks, and example programs. The `include/` directory contains the public interfaces, while `src/` contains the corresponding implementations.

`scripts/` contains the commands used to run the project in different environments. In particular, `run_suite.sh` provides a common procedure for running tests and benchmarks.

`analysis/` contains the Python tools used to analyse the results. These scripts process the raw benchmark results and generate the figures used in the report.

`results/` contains generated files and is not part of the source repository. It is divided according to the type of output: benchmark results, test logs, analysis results, and spectrograms produced by the example programs. Keeping generated files in a separate location avoids mixing experimental data with the source code.

`.github/` together with `system-deps.txt` and `Singularity.def`, describes the project’s development and execution environment. GitHub Actions is used for continuous integration, while the dependency list and the Singularity definition help reproduce the software environment on HPC systems.

This organisation was chosen to keep the project modular and easy to use. The library contains the actual scientific computation, the scripts describe how the software is executed, and the analysis tools describe how the experimental results are processed.

The project also makes use of the header-only `AudioFile` library by Adam Stark for reading and writing audio files. The `AudioFile.h` file used in the project was obtained from Adam Stark’s GitHub repository and is distributed under the MIT License.

4 Design and architecture

4.1 Compile-time polymorphism with CRTP

The project provides three FFT implementations and three window functions with a common interface. Throughout this report, the word *engine* refers to one of these three FFT implementations: `RecursiveFFT`, `IterativeFFT` or `ParallelFFT`. The FFT and window hierarchies use the **Curiously Recurring Template Pattern** (CRTP) to enable static polymorphism and avoid the runtime overhead of dynamic dispatch. This allows common operations to be defined once while keeping each concrete implementation separate.

Listing 1: CRTP base class for the FFT hierarchy (`BaseFFT.hpp`).

```

template <typename Derived>
class BaseFFT {
public:
    void forward(std::vector<std::complex<double>>& data) {
        checkPowerOfTwo(data.size());
        static_cast<Derived*>(this)->forward_impl(data);
    }
};

```

```

    }

    void inverse(std::vector<std::complex<double>>& data) {
        checkPowerOfTwo(data.size());
        static_cast<Derived*>(this)->inverse_impl(data);
    }

protected:
    BaseFFT() = default;
    void checkPowerOfTwo(size_t n) const { /* ... */ }
};

```

Listing 2: Concrete FFT implementation inheriting from BaseFFT<IterativeFFT> (IterativeFFT.hpp).

```

class IterativeFFT : public BaseFFT<IterativeFFT> {
    friend class BaseFFT<IterativeFFT>;

private:
    void forward_impl(std::vector<std::complex<double>>& data);
    void inverse_impl(std::vector<std::complex<double>>& data);
    void bitReverse(std::vector<std::complex<double>>& data) const;
    void butterflyPass(std::vector<std::complex<double>>& data,
                       bool inverse) const;
};

```

The window hierarchy follows the same pattern: BaseWindow<Derived> handles the common operations and stores the coefficients, while each derived class such as HannWindow only implements the corresponding mathematical formula.

4.2 Interface checking with C++20 concepts

The FFT and window interfaces are checked using **C++20 concepts**. This makes the requirements of STFTAnalyzer explicit at compile time and produces clearer errors when an incompatible implementation is used. The concepts therefore provide a simple way to enforce the common interface while allowing new FFT or window implementations to be added.

Listing 3: The IsFFT concept, requiring forward() and inverse() on a complex vector (BaseFFT.hpp).

```

template<typename T>
concept IsFFT = requires(T a, std::vector<std::complex<double>>& data) {
    { a.forward(data) } -> std::same_as<void>;
    { a.inverse(data) } -> std::same_as<void>;
};

```

Listing 4: The WindowFunction concept, requiring construction from a frame size and the operations needed by the STFT analyzer (BaseWindow.hpp).

```

template<typename W>
concept WindowFunction =
    std::constructible_from<W, std::size_t>
    &&
    requires(W w, const W cw, std::vector<double>& sig)
    {
        { cw.size() } -> std::same_as<std::size_t>;
        { w.apply(sig) } -> std::same_as<void>;
        { w(sig) } -> std::same_as<void>;
        { cw.coefficients() } -> std::same_as<const std::vector<double>&>;
        { cw.coherentGain() } -> std::same_as<double>;
        { cw.powerBandwidth() } -> std::same_as<double>;
    };

```

These two concepts are then used to constrain the STFTAnalyzer:

Listing 5: STFTAnalyzer constrained by IsFFT and WindowFunction (STFTAnalyzer.hpp).

```
template<typename FFT, typename Window>
    requires IsFFT<FFT> && WindowFunction<Window>
class STFTAnalyzer {
    // ...
};
```

4.3 FFT factory and independent parallelism levels

The STFT analyzer supports two independent levels of parallelism: across frames and within each FFT. These may require different numbers of threads, so the FFT is created through an **FFT factory** rather than stored as a pre-built object. The factory allows the FFT implementation and its configuration to be selected independently of the STFT analyzer.

Listing 6: The factory type and an example of configuring a ParallelFFT through it (STFTAnalyzer.hpp).

```
using FFTFactory = std::function<FFT()>;

// Default constructor: factory just default-constructs the engine
fftFactory_([] { return FFT(); });

// Caller-supplied factory: fixes ParallelFFT's thread count in advance
STFTAnalyzer<ParallelFFT, HannWindow> a(
    1024, 512, 44100,
    [n = threads] { return ParallelFFT(n); },
    Parallelism::Transform);
```

Listing 7: Each OpenMP thread builds its own private engine from the factory (STFTAnalyzer.hpp).

```
#pragma omp parallel
{
    Window win(frameSize_);
    FFT fft = fftFactory_(); // one engine per thread, never shared
    // ... process this thread's share of the frames ...
}
```

5 Parallelisation strategy

Every STFT frame is independent of every other one: a frame reads its own slice of the signal and writes its own row of the spectrogram, and never touches the data of another. This independence is what makes the computation parallel, and the library turns it into speed at three different levels that stack on top of each other. The first two share memory and stay inside a single machine; the third crosses machines. This section describes how each level is built and how they are composed.

5.1 Level one: parallelism inside a single transform

The finest level lives inside one FFT. ParallelFFT runs the same iterative Cooley–Tukey algorithm as the sequential engine — a bit-reversal permutation followed by butterfly stages of doubling size — but splits the work of each stage across several OpenMP threads, so that the butterflies of a single transform are computed in parallel. This is a shared-memory technique: the threads all read and write the same array, so they must live inside one process and cannot reach past a single node.

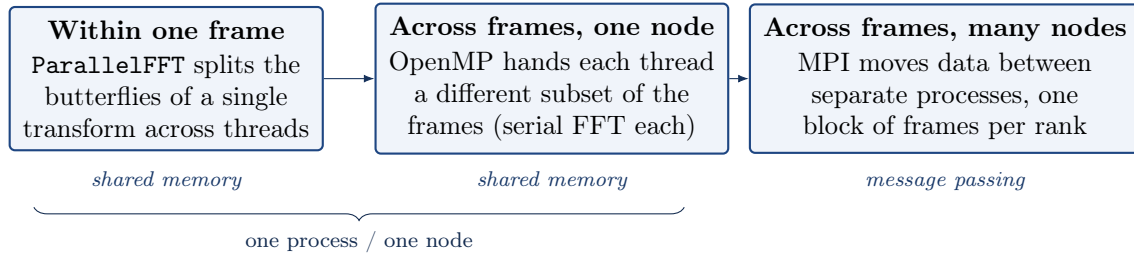


Figure 4: The three levels of parallelism. The first two are shared-memory (OpenMP) and cannot reach past a single node; the third (MPI) is what carries the work across nodes. They compose: an MPI rank can itself use OpenMP inside.

The number of threads is fixed once, when the engine is constructed, and every parallel region names it explicitly rather than reading the ambient OpenMP setting. This makes an engine’s width self-contained: once built for a given number of threads it keeps using that number, even if something elsewhere in the program later changes the global thread count. The stages of the transform are run one after the other because each stage consumes the output of the previous one; only the independent work *within* a stage is spread across threads.

5.2 Level two: parallelism across frames on one node

The second level steps outside the transform and parallelises the frame loop instead. Since the frames of the STFT are independent, an OpenMP region can hand each thread a different subset of them, and each thread computes its frames with an ordinary sequential FFT. Nothing is shared between threads, so no locking or synchronisation is ever needed — each frame writes its own row of the output and no two threads ever write the same one.

The one thing that cannot be shared is the working state: both the window and the FFT engine transform their data in place, so a single shared instance would be corrupted by concurrent use. Each thread therefore builds its own private window, its own FFT engine and its own scratch buffers at the start of the parallel region, and keeps them for the frames it is given. The listing below shows the shape of this: the private objects are created inside the parallel region, and the frame loop that follows is split across the threads.

Listing 8: The frame loop parallelised across threads: each thread owns a private window, engine and buffers, so no state is shared (STFTAnalyzer.hpp).

```

#pragma omp parallel
{
    Window win(frameSize_);
    FFT fft = fftFactory_(); // one engine per thread, never shared
    // ... private work buffers ...

    #pragma omp for schedule(static)
    for (/* each frame in this rank's range */)
        computeFrame(/* ... writes this frame's own row ... */);
}

```

Levels one and two are two different ways of spending the same threads: across whole frames, or inside each transform. They are not used together on the same threads — the frame loop pairs with a sequential engine, while a parallel engine is used with a sequential frame loop — and the analyser selects between them at construction with a plain option, described in Section 5.4.

5.3 Level three: parallelism across nodes with MPI

Neither of the first two levels can reach past one node. OpenMP threads live inside a single process and share its memory, whereas a different node is a separate machine with memory of its own that no thread can simply read. To use more than one node the data must be moved explicitly between separate processes, which is what MPI does.

At this level the frames are partitioned into contiguous blocks, one per MPI rank, and each rank computes only the block assigned to it. The root process reads the WAV file and, at the end, collects the finished rows back into a single spectrogram; in between, every rank runs an ordinary local STFT on its own frames, using whichever of the two shared-memory levels it likes. The next section describes how this MPI layer is put together and how the input is moved to the ranks.

5.4 Composition: hybrid parallelism and the MPI layer

Composing the FFT engine with level two OpenMP parallelism. The two shared-memory levels are exposed by the STFT analyser as a single choice, made when the analyser is built: spend the threads across frames (level two, paired with a sequential engine) or inside each transform (level one, paired with `ParallelFFT`). Both spend the same threads, only in a different place, and the rest of the code does not change between the two.

What makes this switch clean is that the analyser does not hold a ready-made FFT engine. It holds a small **factory** instead — a function that builds a fresh engine on demand. This has two consequences. First, in the across-frames mode the factory is simply called once inside each thread, which is exactly how every thread ends up with its own private engine. Second, it is the only way to build an engine that needs a constructor argument: a `ParallelFFT` must be told how many threads to use, and the factory can capture that number and hand back an engine already configured for it. In other words, the analyser can ask for a `ParallelFFT` with as many threads as it wants without knowing anything about how that engine is constructed.

Listing 9: The engine is built through a factory, so a `ParallelFFT` can be created with a chosen thread count (`STFTAnalyzer.hpp`).

```
using FFTFactory = std::function<FFT()>;

// default: the factory just builds the standard engine
fftFactory_ = [] { return FFT(); };

// configured: the factory fixes ParallelFFT's thread count in advance
STFTAnalyzer<ParallelFFT, HannWindow> analyzer(
    1024, 512, 44100,
    [n = threads] { return ParallelFFT(n); });
```

Composing OpenMP with MPI. The MPI layer is built *around* the shared-memory analyser, not by extending it. `MPI_STFTAnalyzer` does not inherit from `STFTAnalyzer`: it contains one, and hands it the block of frames belonging to this rank. All the actual STFT work is delegated to the contained analyser, which is not even aware that it is running under MPI. The MPI layer is therefore responsible only for two things — distributing the frames among the ranks and collecting the results — while how a rank computes its own block, sequentially or with either shared-memory level, remains entirely the analyser's business. This is what makes the hybrid configuration natural: an MPI rank can run OpenMP across its own frames, so the two levels of parallelism combine simply by composition, with neither layer needing to know about the other.

Listing 10: `MPI_STFTAnalyzer` holds an analyser by composition and delegates the local computation to it (`MPI_STFTAnalyzer.hpp`).

```

class MPI_STFTAnalyzer {
    // ...
    const MPIContext& ctx_;
    STFTAnalyzer<FFT, Window> engine_; // contained, not inherited
    Distribution dist_;
};

// each rank computes only its own block of frames:
SpectrogramData local =
    engine_.analyzeRange(localSignal, localStart, myCount);

```

Input distribution: Broadcast vs Scatter. Once the frames are partitioned, each rank still needs the input samples that its frames read. The library offers two strategies. With **Broadcast** the complete signal is sent to every rank: this is the simpler option, since each process then holds the whole input, but every rank stores a full copy of the signal, so the input memory per rank does not decrease as ranks are added. With **Scatter** each rank instead receives only the samples that its own frames span; consecutive slices overlap slightly, because adjacent frames share some input samples, but that overlap needs no synchronisation because those samples are only read, never written. The scheme below sketches the flow: the root sends out the slices, each rank runs its local STFT, and the finished blocks are collected back on the root.

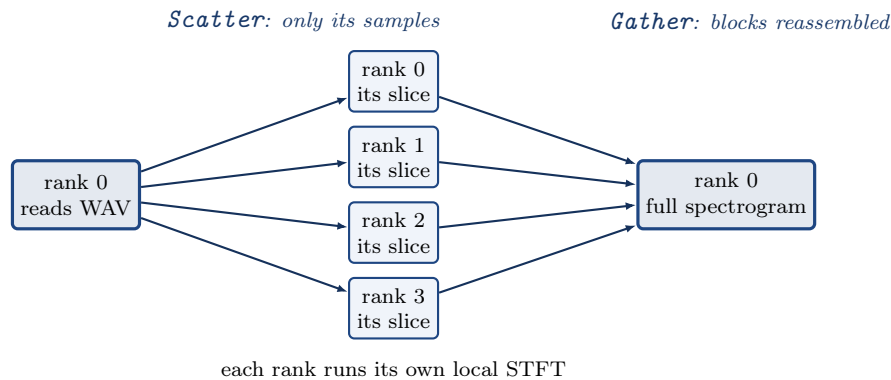


Figure 5: The distributed pipeline. Rank 0 reads the input and reassembles the output, while every rank computes its own block of frames in between. Under **Scatter** each rank receives only the samples its frames span; under **Broadcast** it would receive the whole signal instead.

The difference between the two shows up directly in memory, as Figure 6 illustrates. The upper panel is the input each rank has to hold: under **Broadcast** it stays constant however many ranks are used, since every rank keeps the whole signal, while under **Scatter** it falls as the ranks grow, because each rank keeps only its own slice. The lower panel is the measured peak memory of the whole process, which also includes the spectrogram assembled on the root; this part does not shrink, so the total falls more gently than the input alone, but **Scatter** still uses noticeably less memory per rank across the whole range. This is why **Scatter** is the default strategy.

Managing the MPI environment with RAII. The MPI environment itself is handled with RAII, through a small `MPIContext` object. MPI must be started before any communication and shut down at the end of the run, and forgetting either is a common source of bugs. `MPIContext` ties both to an object's lifetime: its constructor initialises MPI and its destructor finalises it, so creating one at the top of `main()` is all that is needed, with nothing to remember to shut down by hand. It is deliberately neither copyable nor movable, since there is only ever one MPI

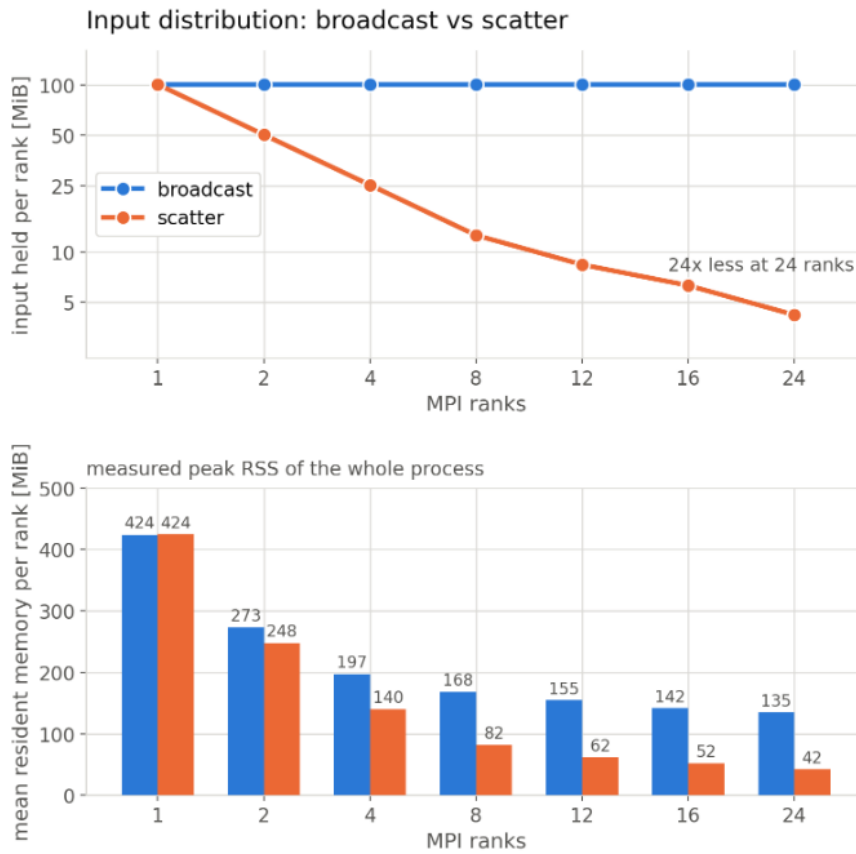


Figure 6: Memory of the two distribution strategies against the number of MPI processes. Top: input bytes held by each rank, computed from the frame layout. Bottom: measured peak resident memory of the whole process. Under **Broadcast** the per-rank input stays constant, while under **Scatter** it shrinks as ranks are added.

environment per process, and it also gives every rank the three facts it needs: its own rank number, the total number of ranks, and whether it is the root.

Listing 11: MPIContext: MPI initialisation and finalisation tied to an object’s lifetime (MPIContext.hpp).

```
class MPIContext {
public:
    MPIContext(int& argc, char**& argv); // calls MPI_Init
    ~MPIContext() noexcept; // calls MPI_Finalize

    MPIContext(const MPIContext&) = delete; // one environment,
    MPIContext(MPIContext&&) = delete; // one owner

    int rank() const noexcept; // this process's rank
    int size() const noexcept; // number of ranks
    bool isRoot() const noexcept; // rank == 0 ?
};

int main(int argc, char** argv) {
    stft::MPIContext ctx(argc, argv); // MPI_Init here
    // ... ctx.rank(), ctx.size(), ctx.isRoot() ...
    return 0; // MPI_Finalize on scope exit
}
```

6 Benchmarks

This section reports the performance campaign carried out on the library.

6.1 Methodology

The benchmark campaign was run on Galileo100 at CINECA (partition `g100_usr_prod`) in exclusive mode: one node for the single-machine questions, four nodes for the distributed ones. Each node has two Intel Xeon Platinum 8260, 48 physical cores with no SMT and 71.5 MiB of last-level cache, so the allocation provides 192 cores. Every point is the mean over 7 repetitions after 2 discarded warm-ups, and the time of an MPI launch is that of the slowest rank (MPI_Reduce with MPI_MAX).

6.2 The three FFT engines

The first experiment isolates the one-dimensional transform from the STFT that surrounds it, so that the cost of the kernel can be measured without the frame loop. A single transform of 2^{18} samples, approximately 5.94 seconds of audio, is computed with each of the three engines, on one MPI process, sweeping the number of OpenMP threads from 1 to 48 (Figure 7).

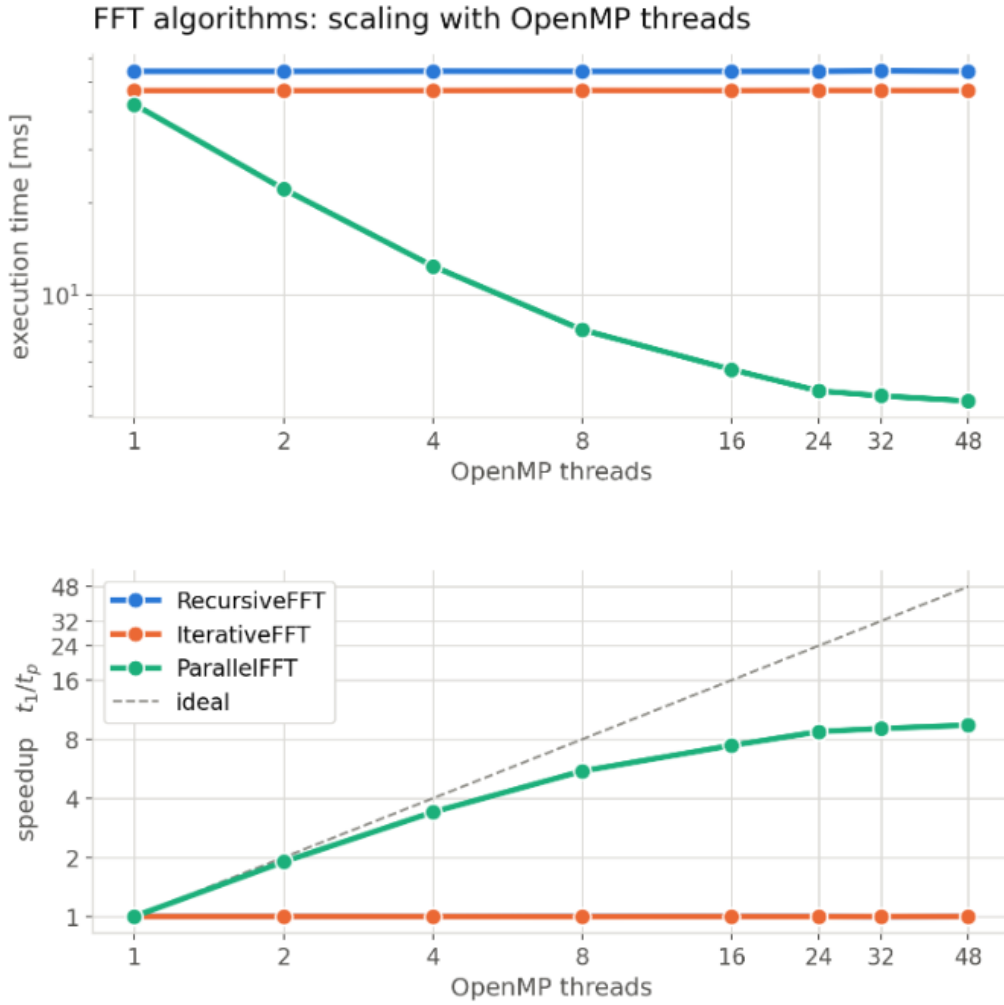


Figure 7: Comparison of the three FFT algorithms as the number of OpenMP threads grows, on a single transform of approximately 5.94 seconds of audio sampled at 44.1 kHz, with one MPI process. Top: execution time in ms (logarithmic scale). Bottom: speedup with respect to the single-thread time of each series.

Two curves are flat, which is the desired outcome: `RecursiveFFT` and `IterativeFFT` are sequential, so the comparison between them is purely algorithmic — removing the recursion is worth some 16% before any parallelization is introduced. `ParallelFFT` is the only engine that responds to the thread count, but it saturates early, most of its gain being already obtained at sixteen threads. The reason is structural: the stages of a Cooley–Tukey transform must be synchronized one after the other, and the work inside a stage does not grow enough to amortize those barriers.

6.3 Where to spend the threads: granularity

This section addresses the following design question: given a fixed budget of threads, is it better to parallelise *across* frames — an OpenMP loop over the frames, each computed with the sequential `IterativeFFT` — or *within* each transform — a sequential loop over the frames, each frame computed with `ParallelFFT`? The two strategies consume exactly the same resources; only the granularity of the tasks differs.

Figure 8 compares them with a budget of 8 workers on a single MPI process, as a function of the audio length (1, 5, 10 and 30 s), with speedup measured against the single-threaded serial

STFT on the same workload. The gap is large and remarkably stable across the four workloads: parallelising across frames lands close to the ideal of 8, while parallelising inside the transform never reaches $1.5\times$. At equal resources the coarse-grained strategy is worth about five times the fine-grained one, the ratio growing slightly with the size of the workload.

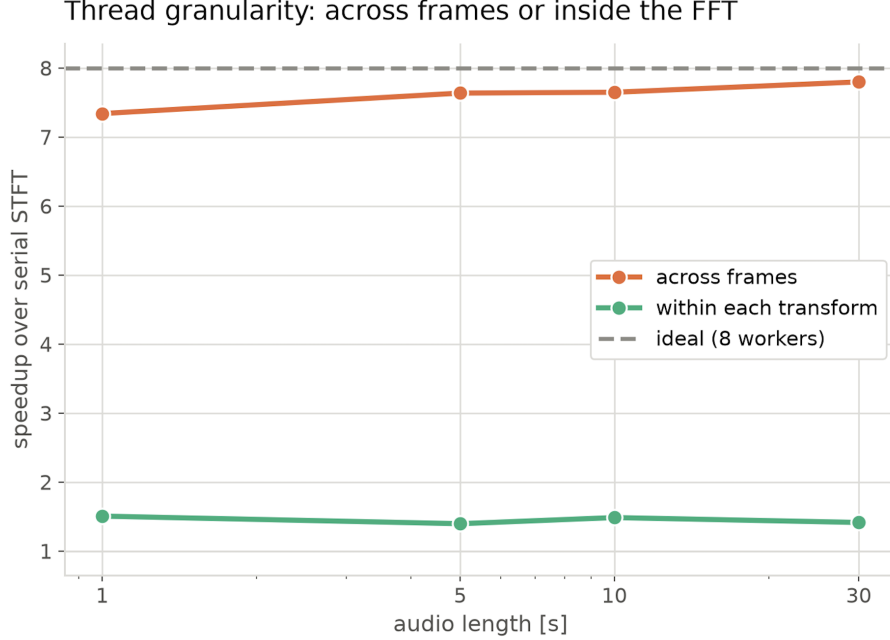


Figure 8: Thread granularity with a budget of 8 workers on one MPI process: parallelising across frames (orange) against parallelising within each transform (green), as a function of the audio length. The ideal line is at 8.

Frames are whole, independent tasks that can be handed to a thread and then work autonomously, whereas the stages of one FFT each consume the output of the previous one and must synchronise at every step. This is what justifies putting OpenMP on the frame loop and using the sequential `IterativeFFT` as the engine of the STFT.

6.4 Strong scaling of the STFT

The next two experiments measure the full STFT pipeline under the two pure execution modes, keeping the workload fixed and increasing the number of workers: first with the parallelism entirely in the threads, then entirely in the processes.

Shared memory. Figure 9 reports the speedup as a function of the number of OpenMP threads on a single MPI process, so that no communication takes place at all. This is the strongest result of the campaign. Up to 8 threads the three workloads sit essentially on the ideal line and only afterwards do they slowly depart from it, still holding an efficiency between 72% and 81% on all 48 threads. The three curves are almost superimposed, which confirms that the behaviour of the frame loop does not depend on the length of the audio: the frames are independent tasks of uniform size and the loop distributes them evenly regardless of how many there are.

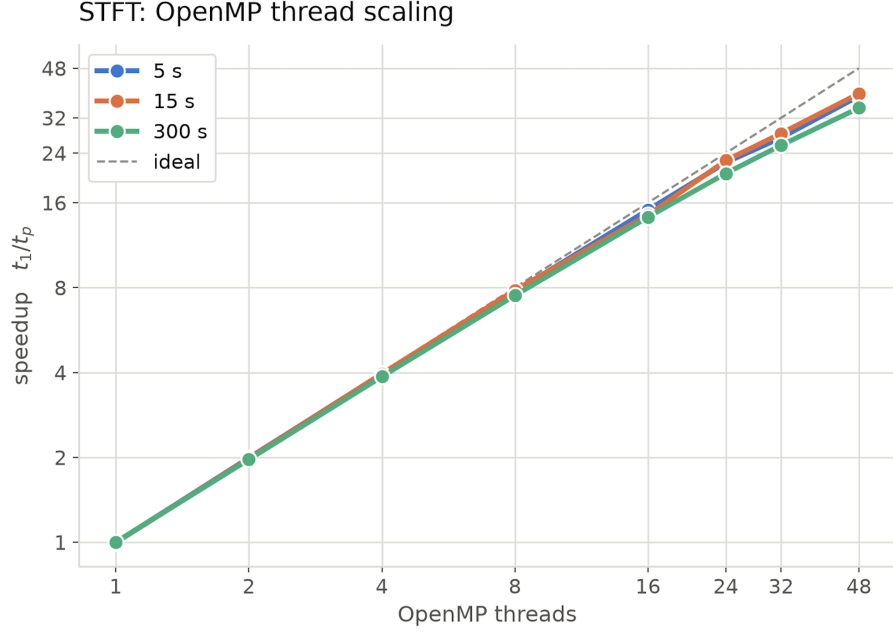


Figure 9: STFT with OpenMP only: speedup as a function of the number of threads (1–48) on a single MPI process. One curve per audio duration, plus the dashed ideal line.

Distributed memory. Figure 10 repeats the experiment with one thread per process, so that all the parallelism comes from the MPI ranks, and extends it across the four nodes of the allocation. With 48 cores per node the rank counts are chosen so that the node boundaries fall on the measured points: 48 processes occupy exactly one node, 96 two, 144 three and 192 four.

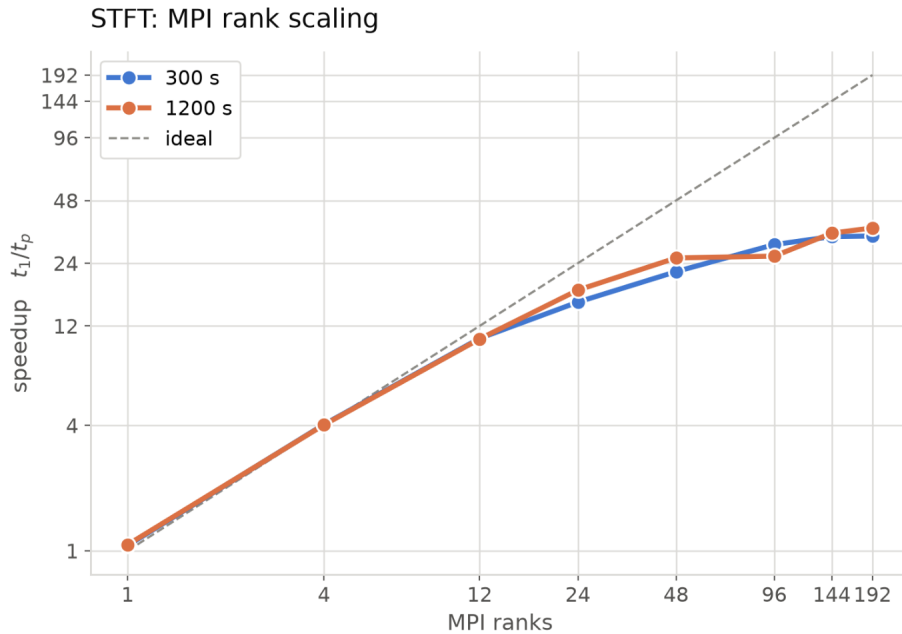


Figure 10: STFT with MPI only: speedup as a function of the number of processes (1–192) with one thread per process, on four nodes. With 48 cores per node, 48 processes occupy one node, 96 two, 144 three and 192 four.

The speedup grows over the whole sweep, reaching $33.7\times$ on 192 processes for the 300 s input,

but the efficiency falls steadily and is below 0.2 at the full allocation. Most of the improvement is obtained within the first node, and the remaining three nodes add comparatively little. The longer input scales better at almost every rank count; this is the expected signature of a fixed communication cost divided over more useful work per rank, as discussed in detail in Section 6.5.

6.5 The limits of the MPI implementation

The efficiency reported above falls as processes are added, but the speedup curve alone cannot say whether the cause is the computation failing to divide or the communication that surrounds it. To separate the two, an analysis was instrumented to time its three phases independently: *distribute*, the `MPI_Scatterv` that moves each process’s slice of the input out of the root; *compute*, the local STFT over that process’s frames; and *gather*, the `MPI_Gatherv` that reassembles the full magnitude matrix on the root, including the allocation of the buffer it lands in.

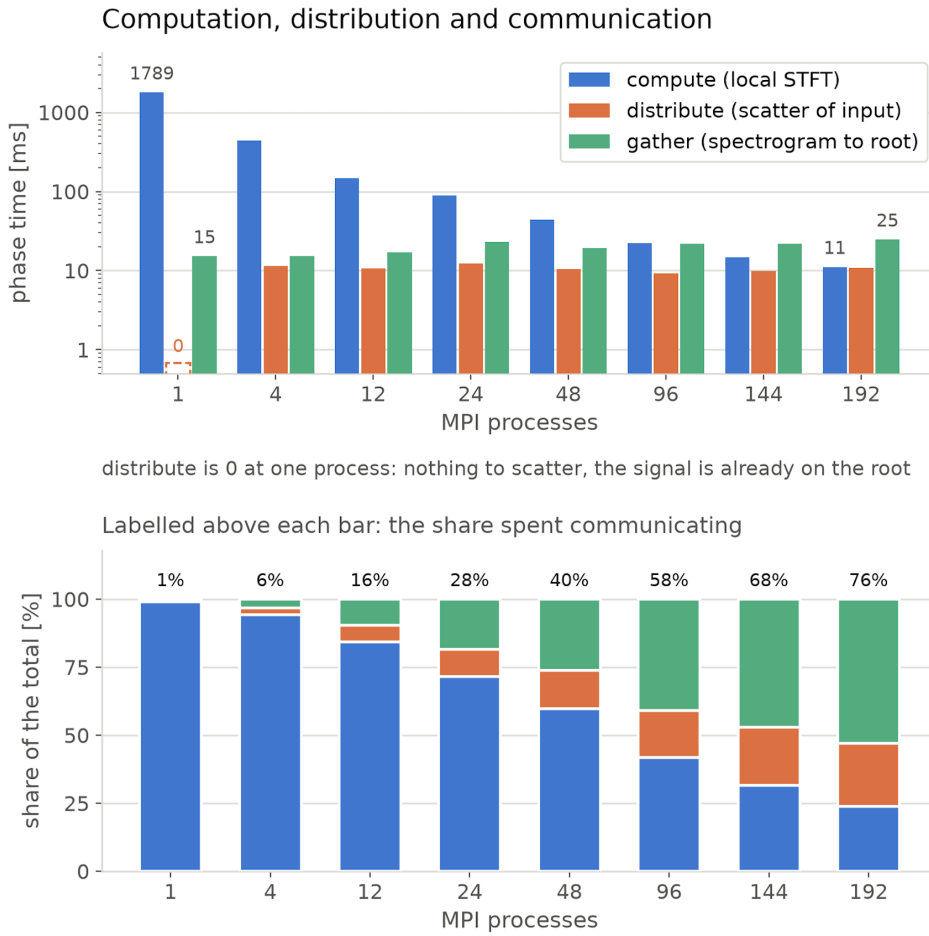


Figure 11: The three phases of the MPI STFT for the 300s workload, with one thread per process and the scatter distribution strategy. Above: wall-clock time per phase, on a logarithmic axis. Below: the same runs as a share of the total, each bar labelled with the fraction spent communicating.

The measurement regards the part of the code that was actually parallelized: the computation divides almost ideally, at an efficiency of 84% on 192 processes, so the loss of scalability does not originate in the STFT. It originates in the two collectives that bracket it, and neither improves as ranks are added. The scatter is flat because it is rooted: whatever the number of processes, the whole signal still has to leave rank 0, whose outgoing bandwidth no process

count changes. The gather is worse, and is the only phase that grows with the process count: every rank sends its results to rank 0 at once, and that single receiver must also allocate and initialise the memory for the full spectrogram while the others wait inside the collective.

Together the two form a floor that no number of processes reduces. The limit is structural: the algorithm is *rooted at both ends*, since the WAV is read by rank 0 and the spectrogram is written by rank 0, while the frames in between never need to meet and could run on any number of ranks. That boundary is the entire difference between the 84% efficiency measured on the computation and the 18% measured on the run as a whole.

A parallel-I/O implementation. Since the floor comes from routing everything through rank 0, the natural fix is to remove that single point: let every rank read and write the file system directly, instead of gathering the result on the root first. Replacing the gather and the scatter with parallel I/O would let the cost be shared across all ranks rather than paid by one, without changing how the STFT work itself is divided. We did not implement this, but it is the clear next step suggested by the phase breakdown above.

7 Conclusions

This work presented `AMSC_STFT`, a modern C++20 library for the Short-Time Fourier Transform, covering the whole pipeline from a WAV file to a time–frequency spectrogram. The library provides three interchangeable FFT implementations, three window functions, and two levels of parallelism — OpenMP for shared memory and MPI for distributed memory.

OpenMP over the frame loop is the strong point of the library: it stays close to the ideal line for most of the thread range and keeps a high efficiency on a full node, almost independently of the length of the audio.

Future work

The phase breakdown of Section 6.5 points directly to that next step. Replacing the scatter and the gather with the collective file operations provided by MPI would let every rank read and write its own portion of the file directly, since each rank already knows which frames, and therefore which bytes, belong to it. This spreads across all ranks a cost that is currently paid by one, and it does so without changing how the STFT work is divided: the two shared-memory levels and the frame partitioning stay exactly as they are. The modular design makes this a localized change, confined to the MPI layer.

Beyond this, the library lends itself naturally to batch spectrogram analysis of large audio archives — music information retrieval, bioacoustics, or machine-condition monitoring — where many long recordings must be turned into images and the shared-memory speed is what counts.

References

- [1] M. Arzeni and M. Colombo, *AMSC_STFT: A parallel C++ library for the Short-Time Fourier Transform*. GitHub repository, 2026.
https://github.com/Maarzeni/AMSC_STFT
- [2] J. O. Smith, *Spectral Audio Signal Processing*. Center for Computer Research in Music and Acoustics (CCRMA), Stanford University, online book, accessed 2026.
<https://ccrma.stanford.edu/~jos/sasp/>
- [3] A. Findelair, *The Short-Time Fourier Transform*. GitHub repository, accessed 2026.
https://github.com/ArthurFDLR/ShortTime_FourierTransform

- [4] L. Formaggia et al., *Advanced Methods for Scientific Computing (AMSC)*. Course material, Politecnico di Milano, 2025/2026.
- [5] ISO/IEC, *ISO/IEC 14882:2020 — Programming Languages — C++ (C++20)*. International Organization for Standardization, 2020. Reference documentation: <https://en.cppreference.com/w/cpp/20>